Explain Last Stone Weight DP

Absolutely. Your handwritten image is showing the **core mathematical transformation** behind LeetCode 1049. The important thing is to understand why a problem that looks like a **simulation of smashing stones** becomes a **0/1 Knapsack / Subset Sum** problem.

1049. Last Stone Weight II

1. What is the problem asking?

You are given stones with different weights.

For example:

```
stones = [2, 7, 4, 1, 8, 1]
```

You repeatedly choose two stones:

- Let their weights be `x` and `y`
- Assume `x <= y`
- After smashing:

```
x == y  → both disappear
x < y   → y - x remains
```

At the end, there can be at most one stone.

We need to return the **minimum possible final stone weight**.

---

2. The first important observation

Suppose we have:

```
[a, b, c, d]
```

There are many possible ways the final answer can be formed.

For example:

```
(a + b) - (c + d)
```

or

```
a - (b + c + d)
```

or

```
(b + d) - (a + c)
```

or

```
(a + b + d) - c
```

etc.

Your image is showing exactly this idea.

The important observation is:

> Every possible final answer can be represented as the difference between two groups of stones.

Let's call them:

```
S1 = one group
S2 = the remaining group
```

So:

```
answer = S1 - S2
```

We can assume:

```
S1 >= S2
```

because the final answer cannot be negative.

3. Why can we think of it as two groups?

This is the key intuition.

Suppose:

```
[2, 7, 4, 1]
```

If we smash **2** and **7**:

```
7 - 2 = 5
```

Then we have:

```
[5, 4, 1]
```

If we smash **5** and **4**:

```
5 - 4 = 1
```

Then:

```
[1, 1]
```

Finally:

```
1 - 1 = 0
```

Mathematically, this sequence is equivalent to:

```
(7 - 2) - 4 - 1
```

which is:

```
7 - (2 + 4 + 1)
```

So the final result is:

```
S1 - S2
```

where:

```
S1 = 7
S2 = 2 + 4 + 1
```

Therefore:

```
answer = 7 - 7
       = 0
```

That's the fundamental transformation.

4. Now use the total sum

Let:

```
TOTAL = sum of all stones
```

Since every stone belongs to either **S1** or **S2**:

```
S1 + S2 = TOTAL
```

Therefore:

```
S1 = TOTAL - S2
```

Our answer was:

```
answer = S1 - S2
```

Substitute **S1**:

```
answer = (TOTAL - S2) - S2
```

Therefore:

```
answer = TOTAL - 2*S2
```

This is exactly the equation you're deriving in your image.

---

5. What are we trying to minimize?

We want:

```
answer = TOTAL - 2*S2
```

TOTAL is fixed.

So the only thing we can control is:

```
S2
```

Look at:

```
TOTAL - 2*S2
```

If **S2** becomes bigger:

```
2*S2 becomes bigger
```

therefore:

```
TOTAL - 2*S2 becomes smaller
```

So:

> **To minimize the final stone weight, we need to maximize S2.**

This is the most important observation.

---

6. But how large can S2 be?

We know:

```
S1 >= S2
```

and:

```
S1 + S2 = TOTAL
```

Therefore:

```
S2 <= TOTAL / 2
```

For example, suppose:

```
TOTAL = 22
```

Then:

```
S2 <= 11
```

Why?

If we had:

```
S2 = 13
```

then:

```
S1 = 22 - 13
   = 9
```

So:

```
S1 - S2
= 9 - 13
= -4
```

But a final stone weight cannot be negative.

Therefore:

```
S2 <= TOTAL/2
```

7. The problem has now changed completely

Originally:

> Smash stones in different orders and find the minimum final stone.

After our mathematical transformation:

> Find a subset of stones whose sum is as close as possible to **`TOTAL/2`**, without exceeding it.

That is a classic:

0/1 Knapsack / Subset Sum problem.

---

8. Why "closest to TOTAL/2"?

Suppose:

```
TOTAL = 22
```

Then:

```
TOTAL/2 = 11
```

Suppose we can create these subset sums:

```
2
5
7
9
10
11
```

The best one is:

```
11
```

because:

```
answer = TOTAL - 2*S2
       = 22 - 2*11
       = 0
```

But suppose **`11`** wasn't possible and the largest possible subset sum was **`10`**.

Then:

```
answer = 22 - 2*10
       = 2
```

So we want:

```
maximum achievable subset sum <= TOTAL/2
```

Exactly what your code calculates.

---

9. Complete example from the actual problem

Take:

```
stones = [2,7,4,1,8,1]
```

Calculate total:

```
TOTAL = 2 + 7 + 4 + 1 + 8 + 1
      = 23
```

Therefore:

```
TOTAL/2 = 11
```

Now we want the largest subset sum **<= 11**.

One possible subset is:

```
{8, 2, 1}
```

Its sum is:

```
11
```

Therefore:

```
S2 = 11
```

The remaining stones are:

```
{7,4,1}
```

Their sum is:

```
S1 = 12
```

So:

```
answer = S1 - S2
       = 12 - 11
       = 1
```

Or using the formula:

```
answer = TOTAL - 2*S2

       = 23 - 2*11

       = 1
```

Therefore:

```
answer = 1
```

10. Connecting this directly to your image

Your image has this flow:

```
              All stones
                  |
                  v
          Divide into 2 groups
             /            \
            /              \
          S1                S2
           \                /
            \              /
             \            /
              Difference
                  |
                  v
               S1 - S2
```

Then you wrote:

```
S1 + S2 = TOTAL
```

Therefore:

```
S1 = TOTAL - S2
```

So:

```
S1 - S2
= (TOTAL - S2) - S2
= TOTAL - 2*S2
```

Then:

```
            TOTAL - 2*S2
                 |
                 |
         maximize S2
                 |
                 v
            S2 <= TOTAL/2
                 |
                 v
     Find closest subset sum
         to TOTAL/2
                 |
                 v
          0/1 Knapsack
```

That's the complete mathematical proof behind your approach.

---

11. Why is this 0/1 Knapsack?

We have:

```
stones = [2,7,4,1,8,1]
```

For every stone we have exactly two choices:

```
          stone
         /     \
      take    don't take
```

For example, for stone `7`:

```
take 7
```

or:

```
don't take 7
```

And importantly:

> We can use each stone at most once.

That's why it is **0/1 Knapsack**.

```
0 → don't take
1 → take
```

12. DP State

Your tabulation uses:

```
C++
dp[i][j]
```

Meaning:

> Using the first `i` stones, what is the **maximum subset sum** we can make that is `<= j`?

For example:

```
dp[3][10]
```

means:

> Using the first 3 stones, what is the largest sum we can create without exceeding 10?

This is a very important distinction.

It does **not** mean:

> Can I make sum 10?

Instead it means:

> What is the maximum sum I can make up to 10?

---

13. Transition

Suppose:

```
stones[i-1] = x
```

For every state `dp[i][j]`, we have two choices.

Choice 1: Don't take the stone

Then:

```
dp[i][j] = dp[i-1][j]
```

We simply ignore the current stone.

Choice 2: Take the stone

If:

```
x <= j
```

then we can take it.

Remaining capacity:

```
j - x
```

So:

```
dp[i-1][j-x]
```

is the best sum we can create using previous stones.

Then add `x`:

```
dp[i-1][j-x] + x
```

Therefore:

```
dp[i][j] =
max(
    dp[i-1][j],
    dp[i-1][j-x] + x
)
```

Exactly what your code does.

---

14. Tabulation code

Your code is already correct. I would write it slightly more cleanly:

```C++
class Solution {
public:
    int lastStoneWeightII(vector<int>& stones) {

        int n = stones.size();

        // 1. Calculate total sum
        int total = 0;
        for (int x : stones) {
            total += x;
        }

        // We only need subset sums up to total / 2
        int target = total / 2;

        // dp[i][j] =
        // maximum subset sum <= j
        // using first i stones
        vector<vector<int>> dp(
            n + 1,
            vector<int>(target + 1, 0)
        );

        // 2. 0/1 Knapsack
        for (int i = 1; i <= n; i++) {

            for (int j = 1; j <= target; j++) {

                // Don't take current stone
```

```
                dp[i][j] = dp[i - 1][j];

                // Take current stone if possible
                if (stones[i - 1] <= j) {

                    dp[i][j] = max(
                        dp[i][j],
                        dp[i - 1][j - stones[i - 1]]
                        + stones[i - 1]
                    );
                }
            }
        }

        // dp[n][target] = largest subset sum <= total/2
        int S2 = dp[n][target];

        // Answer = total - 2*S2
        return total - 2 * S2;
    }
};
```

15. Now let's derive the Memoization version

The tabulation state is:

```
dp[i][j]
```

So our recursive function can naturally be:

```
C++
solve(i, target)
```

Meaning:

> Using stones from index `i` onward, what maximum subset sum can I create without exceeding **`target`**?

At every stone:

```
                    stone[i]
                   /        \
              DON'T TAKE    TAKE
                  |           |
                  v           v
            solve(i+1,j) solve(i+1,j-stone[i])
```

If we take the stone, we add its value:

```
stone[i] + solve(i+1, j-stone[i])
```

Therefore:

```
solve(i,j) =
max(
    solve(i+1,j),
    stone[i] + solve(i+1,j-stone[i])
)
```

16. Memoization code

```
C++
class Solution {
public:

    int solve(int i,
              int target,
              vector<int>& stones,
              vector<vector<int>>& memo) {

        // No stones left
        if (i == stones.size()) {
            return 0;
        }

        // Already calculated
        if (memo[i][target] != -1) {
            return memo[i][target];
        }

        // Option 1: Don't take current stone
        int notTake = solve(
            i + 1,
            target,
            stones,
            memo
        );

        // Option 2: Take current stone
        int take = 0;

        if (stones[i] <= target) {
```

```
            take = stones[i] + solve(
                i + 1,
                target - stones[i],
                stones,
                memo
            );
        }

        return memo[i][target] = max(take, notTake);
    }

    int lastStoneWeightII(vector<int>& stones) {

        int total = 0;

        for (int x : stones) {
            total += x;
        }

        int target = total / 2;

        vector<vector<int>> memo(
            stones.size(),
            vector<int>(target + 1, -1)
        );

        int S2 = solve(
            0,
            target,
            stones,
            memo
        );

        return total - 2 * S2;
    }
};
```

17. Memoization dry run

Take:

```
stones = [2,7,4,1,8,1]
```

We calculated:

```
total = 23
target = 11
```

We call:

```
solve(0,11)
```

Current stone:

```
stones[0] = 2
```

Two choices:

```
                  solve(0,11)
                  /          \
            DON'T TAKE      TAKE 2
               |               |
               v               v
          solve(1,11)      solve(1,9)
```

For **`solve(1,9)`**:

```
current = 7
```

Again:

```
              solve(1,9)
              /         \
        DON'T TAKE       TAKE 7
           |               |
           v               v
      solve(2,9)        solve(2,2)
```

And so on.

Eventually the recursion discovers:

```
2 + 8 + 1 = 11
```

So:

```
S2 = 11
```

Then:

```
answer = 23 - 2*11
       = 1
```

18. Memoization vs Tabulation

This is something worth understanding deeply because you've been working on converting memoization into tabulation.

Memoization

Starts from the final question:

```
solve(0, target)
```

and recursively goes:

```
(i,target)
    |
    +---- take
    |
    +---- don't take
```

Only states that recursion reaches are calculated.

Tabulation

Instead of starting from:

```
solve(0,target)
```

we build the answers from smaller states:

```
dp[0][0...target]
```

then:

```
dp[1][0...target]
```

then:

```
dp[2][0...target]
```

and so on.

So:

```
Memoization:

        solve(0,target)
             |
       --------------
       |            |
      take       don't take
       |            |
       v            v
    smaller      smaller
     states       states


Tabulation:

dp[0][...]

      ↓

dp[1][...]

      ↓

dp[2][...]

      ↓

dp[3][...]

      ↓

   ...

dp[n][target]
```

Both are solving the **same state definition**.

---

19. Why do we use `total / 2` instead of `total`?

This optimization is very important.

Suppose:

```
total = 23
```

We only care about:

```
S2 <= 11
```

There is absolutely no reason to calculate subset sums from:

```
12 → 23
```

because those would make **`S2 > S1`**.

Therefore:

```
C++
int target = total / 2;
```

dramatically reduces the DP size.

---

20. Complexity

Let:

```
n = number of stones
S = total sum of stones
```

Our target is:

```
S/2
```

Therefore the 2D DP has approximately:

```
n × S/2
```

states.

Time

```
O(n * S)
```

Space

For the 2D implementation:

```
O(n * S)
```

More precisely, the target is **`S/2`**, but Big-O ignores the constant **`1/2`**.

---

21. The entire intuition in one chain

This is the part I would recommend remembering for interviews/LeetCode:

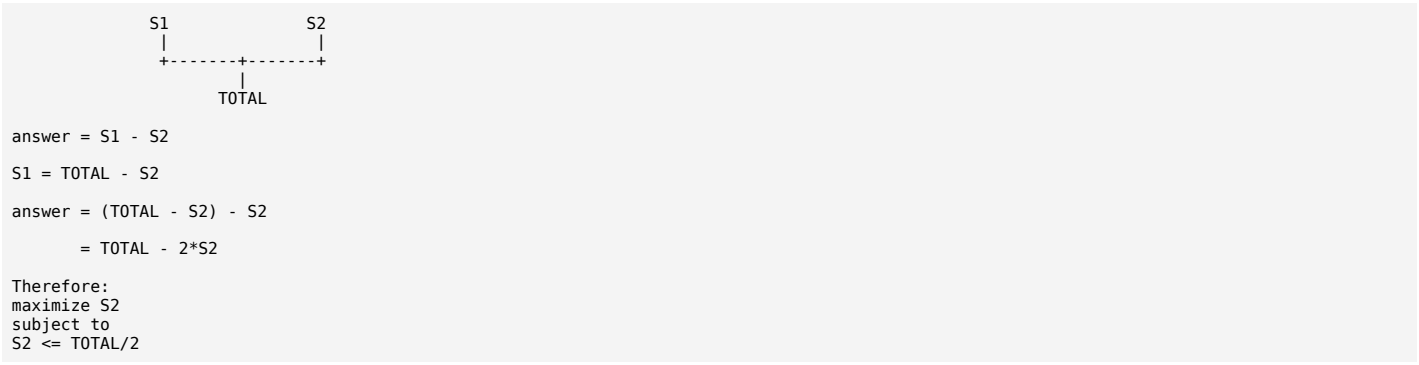
```
                  LAST STONE WEIGHT II
                          |
                          v
              Smashing stones repeatedly
                          |
                          v
            Final answer can be represented
                as difference of two groups
                          |
                          v
                   answer = S1 - S2
                          |
                          v
                   S1 + S2 = TOTAL
                          |
                          v
                   S1 = TOTAL - S2
                          |
                          v
              answer = TOTAL - 2*S2
```

```
                          |
                          v
            TOTAL is fixed, therefore
            maximize S2
                          |
                          v
                 But S2 <= TOTAL/2
                          |
                          v
         Find maximum subset sum <= TOTAL/2
                          |
                          v
                 SUBSET SUM PROBLEM
                          |
                          v
                  0/1 KNAPSACK
                          |
                          v
             dp[i][j] = maximum sum
             <= j using first i stones
                          |
                          v
                S2 = dp[n][TOTAL/2]
                          |
                          v
             answer = TOTAL - 2*S2
```

The one-line intuition

> **Turn the stones into two groups whose sums are as close as possible; if the total is `T`, we want one group's sum as close as possible to `T/2`, which is exactly a 0/1 Knapsack problem.**

And your image is essentially proving this transformation:



That's the complete reason why the seemingly complicated **stone-smashing problem becomes a subset-sum DP problem**.

Yes — **absolutely**. In fact, if **`totalSum`** is even and the stones can be divided into two groups with exactly equal sums, then the maximum subset sum **will be exactly `totalSum / 2`**.

That's actually the **best possible case**.

Example

Suppose:

```
stones = [2, 4, 6, 8]
```

Total:

```
totalSum = 2 + 4 + 6 + 8 = 20
```

So:

```
target = totalSum / 2 = 10
```

Can we make a subset with sum **`10`**?

Yes:

```
{2, 8} = 10
```

The remaining stones are:

```
{4, 6} = 10
```

So:

```
S1 = 10
S2 = 10
```

Therefore:

```
answer = S1 - S2
       = 10 - 10
       = 0
```

Using our formula:

```
answer = totalSum - 2*S2
       = 20 - 2*10
       = 0
```

So **the answer can absolutely be zero**.

Why do we say "as close as possible"?

Because **`totalSum / 2` may or may not be achievable**.

There are two cases.

Case 1: Target is achievable

```
totalSum = 20
target = 10
```

Suppose:

```
stones = [2, 8, 4, 6]
```

We can make:

```
2 + 8 = 10
```

Therefore:

```
max subset sum <= target
= 10
```

So:

```
answer = 20 - 2(10)
       = 0
```

Case 2: Target is NOT achievable

Suppose:

```
stones = [2, 7, 4]
```

Then:

```
totalSum = 13
target = 13/2 = 6
```

Possible subset sums up to **`6`**:

```
0
2
4
6
```

Actually **`6 = 2 + 4`**, so here target **is achievable**:

```
S2 = 6
```

Then:

```
answer = 13 - 2(6)
       = 1
```

Notice something interesting:

Because **`totalSum`** is odd, **`totalSum / 2`** using integer division is **`6`**.

The two groups become:

```
S2 = 6
```

```
S1 = 7
```

So the minimum possible difference is:

```
7 - 6 = 1
```

The important point

Your DP is **not trying to get something merely "near" the target if the target itself is possible**.

It is doing:

```
Find maximum subset sum <= target
```

Therefore:

```
                target
                  |
        ┌─────────┴──────────┐
        |                    |
  achievable             impossible
        |                    |
        ↓                    ↓
  take target          take closest
        |                smaller sum
        ↓                    ↓
    answer 0          minimum difference
```

So if:

```
target = 10
```

and DP discovers:

```
10
```

then that's automatically the maximum possible value. **There is no reason to search for 9, 8, etc.**

Why can't we go ABOVE the target?

This is the subtle part.

Suppose:

```
totalSum = 20
target = 10
```

Imagine we choose:

```
S2 = 11
```

Then:

```
S1 = 20 - 11
   = 9
```

Now:

```
S1 - S2
= 9 - 11
= -2
```

But we don't want **`S2 > S1`**. We can simply call the larger group **`S1`** instead.

Then the groups are effectively:

```
S1 = 11
S2 = 9
```

and the answer is:

```
11 - 9 = 2
```

That's exactly why we only search up to:

```
totalSum / 2
```

We want the **smaller group**, `S2`, to be as large as possible without becoming larger than the other group.

So remember this exact statement:

> **We need the largest achievable subset sum `S2` such that `S2 <= totalSum/2`. If `totalSum/2` itself is achievable, then we take it, and the answer becomes `0` when the total is even.**

And this is why your DP target is:

```C++
int target = total / 2;
```

and the final answer is:

```C++
return total - 2 * dp[n][target];
```

The DP automatically handles both situations:

```
target achievable     → dp = target
target not achievable → dp = closest smaller sum
```

That's the whole reason for the phrase **"closest to `totalSum/2`."**

Exactly — **this is the important distinction**:

You might think:

> "Just calculate **`total / 2`**, and subtract it from the remaining sum. Why do we need DP?"

Because **`total / 2` is only the ideal target. It is NOT necessarily a sum that we can actually create using the stones.**

We need to find a **subset of actual stones** whose sum is as close as possible to **`total/2`**.

---

1. Suppose the total is 20

```
stones = [2, 8, 4, 6]
```

Total:

```
total = 20
```

Half:

```
total / 2 = 10
```

Now fortunately, we can actually make **`10`**:

```
2 + 8 = 10
```

Remaining:

```
4 + 6 = 10
```

So:

```
S1 = 10
S2 = 10

answer = 10 - 10 = 0
```

Here, yes — we found the perfect half.

---

2. But what if we cannot make the half?

Consider:

```
stones = [8, 7, 5]
```

Total:

```
total = 20
```

Half:

```
target = 10
```

Now ask:

> Can I select some stones whose sum is exactly 10?

Possible subset sums:

```
8
7
5
8 + 7 = 15
8 + 5 = 13
7 + 5 = 12
8 + 7 + 5 = 20
```

There is **no subset with sum 10**.

So we cannot simply say:

```
S2 = 10
```

because **there is no physical way to choose stones totaling 10**.

The closest achievable sum below 10 is:

```
S2 = 8
```

Then the remaining stones are:

```
S1 = 7 + 5
   = 12
```

Therefore:

```
answer = 12 - 8
       = 4
```

Or:

```
answer = total - 2*S2
       = 20 - 2*8
       = 4
```

3. This is exactly why we need subset sum

Think about the difference between these two statements:

Mathematical ideal

```
I WANT:

S2 = total / 2
```

Actual problem

```
CAN I SELECT SOME STONES
WHOSE SUM = total / 2?
```

Those are completely different questions.

For:

```
[8, 7, 5]
```

we want:

```
10
```

but the stones simply **cannot produce 10**.

That's what the DP finds out.

---

4. Think of it like this

Imagine you have these stones:

```
8     7     5
```

You want to divide them into two groups as equally as possible.

You would LIKE:

```
10 | 10
```

But can you actually divide them that way?

No.

The possible divisions are:

```
8 | 7+5
```

which gives:

```
8 | 12
```

Difference:

```
12 - 8 = 4
```

So the real problem is:

> **Find a subset whose sum is as close as possible to half of the total.**

---

5. Why not simply use `total/2`?

Because you're not allowed to invent a new stone.

For example:

```
stones = [8,7,5]
```

You cannot say:

```
S2 = 10
```

because there is no combination:

```
8
7
5
8+7
8+5
7+5
8+7+5
```

that equals **`10`**.

The subset has to be made from the **actual stones**.

That's the entire reason for the DP.

---

6. And "remaining sum" comes AFTER finding the subset

This is the correct order:

```
             All stones
                 |
                 v
            total = 20
                 |
                 v
            target = 10
                 |
                 v
       Can I make subset = 10?
              /       \
            YES        NO
             |          |
             |          v
             |      Find largest
             |      subset < 10
             |          |
             +----------+
                    |
                    v
              S2 = 8
                    |
                    v
```

```
          Remaining sum = 20 - 8
                    |
                    v
                S1 = 12
                    |
                    v
             12 - 8 = 4
```

So we **cannot calculate the remaining sum first**, because we don't yet know which stones belong to the first subset.

---

7. The key sentence to remember

> **`total/2` tells us the ideal sum we want. DP tells us whether the actual stones can produce that sum, and if not, what the closest achievable sum is.**

So:

```cpp
target = total / 2;
```

does **not** mean:

> "The subset sum is **`total/2`**."

It means:

> "Try to find the largest achievable subset sum that does not exceed **`total/2`**."

That's why:

```cpp
dp[n][target]
```

is necessary.

Then only after finding that subset:

```cpp
S2 = dp[n][target];
```

we calculate:

```cpp
answer = total - 2 * S2;
```

In one line:

```
Half = what we WANT
DP   = what we CAN actually make
```

That's the core intuition behind the entire solution.